Sajith.M
24BCE2284

# DBTHON 2026: Project Proposal Report

Database Systems - Assessment 7

## 1. Domain

Enterprise E-Commerce Systems, Financial Accounting, and Advanced Database Engineering.

The project operates at the intersection of high-concurrency online retail platforms and secure transaction auditing, focusing specifically on how relational database engines manage temporal data evolution and cryptographic immutability.

## 2. Problem Statement

Modern e-commerce platforms handle extensive product catalogs, dynamic pricing structures, inventory levels, and financial settlements. Traditional relational database implementations rely on destructive in-place UPDATE operations (e.g., modifying a product price or inventory count directly). While functional for basic CRUD applications, this conventional approach creates critical architectural vulnerabilities in financial accounting, auditing, and temporal data reconstruction. Specifically, overwriting records destroys historical states, preventing systems from accurately answering retrospective queries or verifying financial integrity against post-facto tampering.

## 3. Existing Approach and Limitations

Traditional Approaches

1) Destructive In-Place Updates: Systems overwrite rows when prices or attributes change. Historical context for past orders is either lost or poorly approximated by duplicating columns across transaction tables.
2) Application-Level Temporal Logic: Developers write complex application code and extra junction tables to track price changes over time, leading to normalization violations and maintenance overhead.
3) Mutable Audit Logs: Standard database audit tables or logs can be modified or deleted by users with elevated database privileges, leaving no cryptographic proof of data tampering.

## 4. Core Limitations

1) Lack of Point-in-Time Accuracy: Inability to efficiently query the exact database state or product price at an arbitrary historical timestamp without heavy application-layer joins.
2) Audit Vulnerability: Absence of engine-level cryptographic guarantees to prove that financial transaction records have not been altered since creation.
3) Normalization Trade-offs: Standard workarounds often violate Third Normal Form (3NF) principles by introducing data redundancy.

## 5. Proposed Database Solution

This project introduces an engine-level database architecture built on two primary pillars:

1) Bi-Temporal System-Versioned Price History:

Instead of overwriting product prices, price modifications are recorded as non-destructive temporal intervals using explicit time boundaries (valid_from and valid_to). This enables native database point-in-time queries and stored procedures to retrieve historical pricing instantly.

2) Append-Only Cryptographic Audit Ledger:

Financial transactions and order settlements are recorded in a ledger table where each entry is cryptographically chained to its predecessor using SHA-256 hashes (prev_hash, curr_hash) generated natively via PostgreSQL triggers (pgcrypto). Any unauthorized modification to past records breaks the cryptographic hash chain instantly.

3) Normalized 3NF Schema Architecture:

A fully normalized relational schema consisting of at least 8 interconnected tables (users, categories, products, product_price_history, orders, order_items, ledger_transactions, inventory_logs), enforcing strict primary keys, foreign keys, and integrity constraints (CHECK, UNIQUE, NOT NULL).

## 6. Innovation and Novelty

Unlike standard e-commerce projects that focus solely on basic CRUD operations and UI features, this project derives its novelty from advanced database engine implementation:

- Database-Driven Temporal Integrity: Shifting time-travel query logic and temporal validation directly into PostgreSQL stored procedures and range indexes rather than relying on external caching or microservices.
- Native Cryptographic Immutability: Implementing blockchain-style SHA-256 hash chaining directly inside relational database triggers to guarantee tamper-proof financial ledgers.
- Unified RBAC Security Model: Managing multi-tier access (CUSTOMER, MANAGER, ADMIN) through a single normalized users table with database-enforced role constraints.

## 7. Proposed Technical Approach

Architecture Overview

- Database Management System: PostgreSQL (utilizing native range types, indexing, stored procedures, triggers, and the pgcrypto extension).
- Backend Framework: Express.js (Node.js) handling RESTful API routing, business logic, and JWT authentication.
- ORM Layer: Prisma ORM for type-safe database interactions, schema migrations, and built-in SQL injection prevention.
- Frontend Interface: Next.js (React / TypeScript) providing responsive user dashboards, catalog filtering, and admin time-travel visualizers.
- Security & DevOps: BCrypt password hashing, secure .env secret management, Git version control (main/dev branches), and native pg_dump/pg_restore backup automation.

Sajith.M
24BCE2284

## 8. Sustainable Development Goals (SDG) Relevance

SDG 9: Industry, Innovation, and Infrastructure: Advances resilient database engineering practices by introducing secure, highly audit-compliant data architectures for digital commerce infrastructure.